

PRACTICAL : 5

AIM:- Write a class with four static functions: add, subtract, multiply and divide. Write a test class to take two integer inputs from user and call the static functions for performing mathematical operations.

THEORY :-

1] Explain static variables and static functions with example.

In Java, static members are those which belongs to the class & you can access these members without instantiating the class.

Static Variables :-

You can create a static variable by using the keyword 'static'.

The static variables have the same value in all the instances of the class.

These are created and initialized when the class is loaded. Just like static methods you can access static variables using the class name.

Example: `public class MyClass`

{

`public static int data = 20;`

Date : / / 20

```
public static void main(String[] args)
{
    System.out.println("MyClass.data");
}
```

2] Static Functions/Methods.

You can create a static method by using keyword static. Static methods can access static fields, methods.

To access static methods, there is no need to instantiate the class. You can do it just using the class name.

```
Ex. :- public class MyClass {
        private static int x = 1;
        private int y;

        public static void sample() {
            S.O.P("Hello From Static Function")
            x = 10;
            System.out.println("x = " + x);
        }

        public void sayHello() {
            System.out.println("Hello from regular function");
        }
    }

    public class MyClassTest {
        public static void main(String[] args) {
            MyClass.sample();
            MyClass c1 = new MyClass();
            c1.sayHello();
        }
    }
```


Date: / /20

PRACTICAL : 7

AIM:- Inheritance of Function overriding.

THEORY:-

1] Explain the Inheritance with Example.

Inheritance is a feature of object oriented programming through which we can create new classes from existing classes. This ~~is~~ increases the reusability of classes.

The existing class from which we create new class is called the parent class or superclass or existing class.

The new class i.e. created is called a sub-class or a child class. The subclass inherits all the properties/features of the superclass. The basic syntax for creating a subclass from a superclass as follows

Syntax:

```
public class SubclassName extends
    SuperclassName
{
}
```

Example :-

```
public class Manager extends Employee
{
}
```


Date: / /20

2] Explain Function Overriding with example

When the derived class has some function as that of superclass, it is called function overriding.

The signature of function in derived class, should be same as that of base class.

The derived class function overrides the superclass function. when we call the overridden function with subclass object always the function of subclass is called.

Example :-

```

class cat {
    public void sound () {
        System.out.println("meow");
    }
}

class lion extends cat {
    public void sniff () {
        System.out.println("snif");
    }
    public void sound () {
        System.out.println("roar");
    }
}
    
```

Function overriding. same method name (sound()) & same parameters.

Date: / / 20

PRACTICAL : 8

AIM:- The use of Final method & Final class, abstract methods in Java

THEORY:-

I] Explain the use of Final Keyword.

(a) For avoiding overriding.

If any superclass function should not be overridden in subclasses in that case the superclass functions can be declared as Final

Any function of superclass i.e declared as final cannot be overridden in subclasses.

(b) Final Keyword is also used to avoid inheritance, any class i.e declared as final cannot be inherited.

Ex:- ~~public final class A~~

{
}

public class B extends A // error

// class A is declared as Final

{
}

Date: / / 20

Final key word is also used to declare final variables. Final variables need to be initialized when they are declared. The values of final variables cannot be changed throughout the program.

Ex:- `final int ARRAY_SIZE = 100;`
`final float PI = 3.14;`

2] Explain Abstract methods and classes with example

a) Abstract Methods:-

Any method can be declared as abstract in a class when a method is declared as abstract each does not have implementation & it is known as abstract method.

Ex:- `abstract void printStatus();`

here, `printStatus` Function is declared as abstract using key word & hence it does not have any implementation.

b) Abstract Classes

Any classes having abstract method should be declared as abstract class. A class declared as abstract can have abstract & non-abstract methods.

An abstract class cannot be instantiated. It can ^{have} constructors & static methods. It can also have final methods.

An abstract class can be extended (subclasses can be created

↳

Date: / /20

from abstract class).

All the subclasses of an abstract class should implement all the abstract methods declared in the superclass.

Example:-

```

abstract class A {
    abstract void printStatus();
}
public class B extends A
{
    void printStatus()
    {
        System.out.println("Abstract Method Implementation
        in Subclass");
    }
}
// Test class //
public class Test Abstract
{
    public static void main (String[] args)
    {
        B b1 = new B;
        b1.printStatus();
    }
}

```

OUTPUT:- Abstract Method Implementation in Subclass.